

Cx1106

Computer Organisation and Architecture

Computer Arithmetic related consideration during coding

Prof Lin WeiSi
School of Computer Science and Engineering, NTU
Email: wslin@ntu.edu.sg





Effects of four operators

- Addition/Subtraction
 - Addition/Subtraction will cause the result to increase/decrease
 - When done sufficient number of times, **overflow** at Min or Max end of the representable range will eventually occur
 - Take note of the range of the **data type** used when coding in High level language, or the **width of registers and memory** when coding in assembly
- Multiplication
 - Multiplication in binary is similar to an **arithmetic left shift**
 - **Overflow** at Min or Max end of the representable range
 - Similar considerations as Add/Sub.
- Division
 - Division in binary is similar to an **arithmetic right shift**
 - Truncation of LSB leads to **loss in precision**
 - **Reduces the magnitude** of the result so it get further away from the Min/Max of the range.

Effects of Rounding

- Rounding refers to **removing of LSB(s)** so that the result can fit into the representable bits.
 - The limits imposed in the width of the representable bits could be from the **registers width, data type etc.**
- Rounding can be round-up or round-down to nearest representable number.
 - As the rounded number is an approximation of the raw result, a certain amount of **rounding error is incurred.**
- Below an example of rounding off a floating point number to 2 decimal points, incurring an **error of 0.004×10^1**

1 • 6 8 6 e + 0 1  1 • 6 9 e + 0 1

- Common to have **intermediate register with width larger than regular data registers** to allow intermediate processing to be done at **higher precision** and thus **reducing the amount of rounding error.**

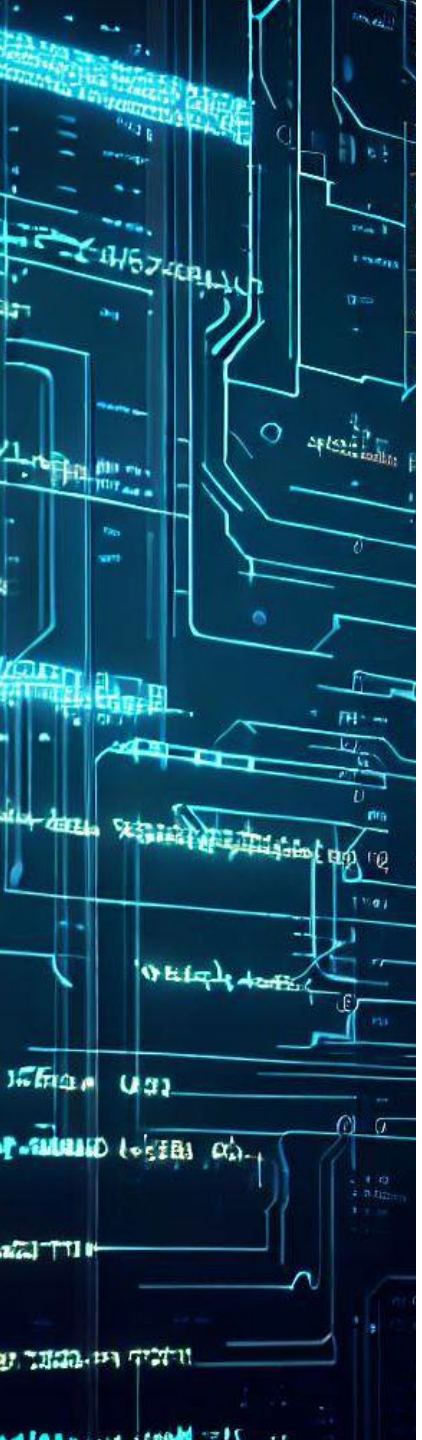
Handling numbers with different magnitudes

Example:

$$\begin{aligned} &1.00 \times 10^0 + 1.00 \times 10^0 + \\ &1.00 \times 10^0 + 1.00 \times 10^0 + \\ &1.00 \times 10^0 + 1.23 \times 10^3 \end{aligned}$$

- The **order of evaluation** can affect accuracy of result
 - Add/subtract operands with **similar size of magnitude** first.

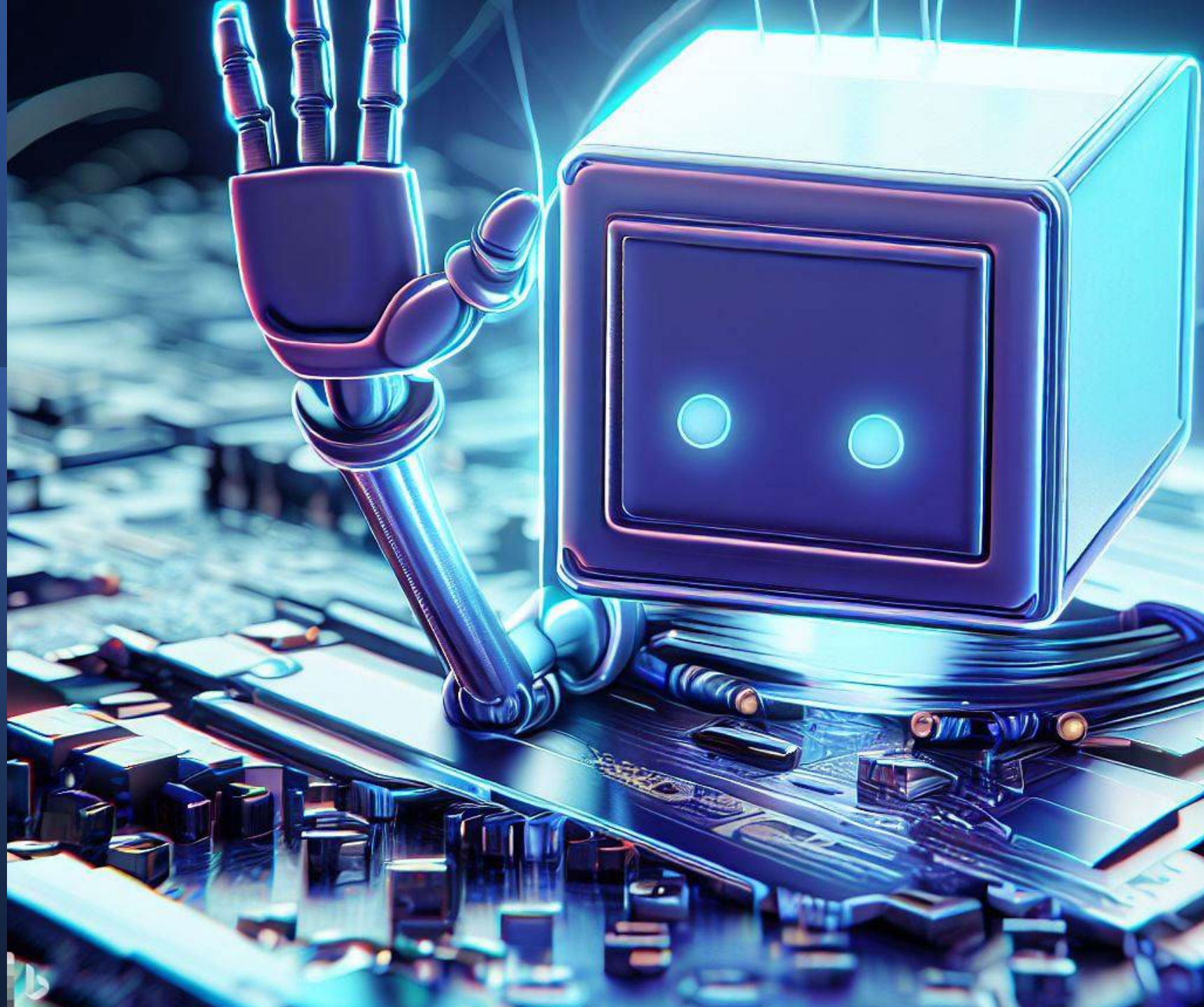
1	•	0	0	0	e	+	0	0	
+	1	•	0	0	0	e	+	0	0
2	•	0	0	0	e	+	0	0	
+	1	•	0	0	0	e	+	0	0
3	•	0	0	0	e	+	0	0	
+	1	•	0	0	0	e	+	0	0
4	•	0	0	0	e	+	0	0	
+	1	•	0	0	0	e	+	0	0
0	•	0	0	5	e	+	0	3	Align
+	1	•	2	3	0	e	+	0	3
1	•	2	4	0	e	+	0	3	Rounding



Maximising Accuracy during computation

- Two issues: **overflow** and **precision**.
- From previous slides, **addition, subtraction and multiplication** may potentially lead to **result overflowing** the range of the representable number used.
- **Division** will lead to **loss in precision** as bits are lost due to truncation of LSB(s).
- **Some rule of thumb**
 - **Accumulate/subtract** numbers with **small magnitude first** to allow their magnitude to be comparable to big magnitude numbers.
 - Take note of the range of the number system used, which, depending on the usage scenario, can be a factor of the **data type, number format, register/memory width** etc.
 - **check for overflow** in a process if possible.
 - To **preserve** as much **precision** as possible, always **do division last** as far as possible.

Thank you



No part of this material shall be filmed, recorded, downloaded, reproduced, distributed, republished, or transmitted in any form or by any means without written approval from Nanyang Technological University.